Table 1: Summary of Key Terms and Definitions

Term	Definition	Example
Algorithmic Primitive	A minimal computational operation observed in a reasoning process	Retrieving the nearest neighbor in TSP
Algorithmic Tracing	The process of identifying all relevant primitives for implementing a particular reasoning process	Solving TSP requires distance calculation, nearest-neighbor selection, path comparison, ...
Primitive Vector	A direction in activation space that can be injected into the residual stream to reliably induce a particular primitive	A vector that causes the model to use more nearest-neighbor heuristics when injected
Primitive Induction	Modifying model behavior by injecting a primitive vector	Injecting a nearest-neighbor primitive vector

Definition: Algorithmic Tracing. We define *algorithmic tracing* as the process of identifying all relevant primitives for implementing a particular reasoning process. By targeting the computational building blocks rather than just behavioral outcomes, algorithmic tracing helps us better understand how and why a model solves a particular task and what other tasks it might be able to solve.

Definition: Primitive Vector. We define primitive vectors (following the function and steering vector literature (Todd et al., 2024)) as vectors that can be injected into the residual stream to reliably induce a particular primitive. For a given model input x, which here refers to the input prompt and the model response, primitive p and transformer layer ℓ, let the residual stream activation at token t be $h_\ell(x,t) \in \mathbb{R}^{d_\ell}$. A *primitive vector* $v_\ell^{(p)} \in \mathbb{R}^{d_\ell}$ is a direction in activation space that increases the expression of the primitive function p. Here we extract primitive vectors using the function vector approach (Todd et al., 2024) (see Section 4.2).

Definition: Primitive Induction for Algorithmic Steering and Causal Evaluation. Given a primitive vector injection strength $\alpha \in \mathbb{R}$, we can intervene on a representation by adding (or patching) a primitive vector: $\tilde{h}_\ell(x,t) = h_\ell(x,t) + \alpha\, v_\ell^{(p)}$, which increases the probability of expressing p when $\alpha > 0$, and decreases it when $\alpha < 0$. We refer to this procedure as *algorithmic steering* or *induction*.

Algebraic Operations on Primitive Vectors. Assuming compositional geometry, primitive vectors can be combined through simple algebraic operations in activation space. For primitives p and q at layer ℓ, additive and subtractive composition within a layer (+ and –) can be defined as $v_\ell^{(p \oplus q)} \approx w_p\, v_\ell^{(p)} \pm w_q\, v_\ell^{(q)}$. Scalar modulation (with varying strength) is defined as $v_\ell^{(\alpha p)} = \alpha\, v_\ell^{(p)}$.

Primitive Transfer: Cross-Task Generalization. We test primitive generalizability by examining whether primitives identified in one domain transfer to others; specifically by examining whether injecting primitives extracted from task 1 has a predictable effect on model performance in task 2. We formalize cross-task transfer as follows. Let $p \in \mathcal{P}_{T_1}$ denote a primitive extracted from source task T_1, with vector $v_{\ell,T_1}^{(p)} \in \mathbb{R}^{d_\ell}$. When evaluating target task T_2, we inject this vector into the residual stream:

$$\tilde{h}_\ell(x_{T_2},t) = h_\ell(x_{T_2},t) + \alpha\, v_{\ell,T_1}^{(p)}.$$

If this intervention increases the activation $a_{\ell,T_2}^{(p)}(x_{T_2},t) = \langle v_{\ell,T_1}^{(p)},\, h_\ell(x_{T_2},t)\rangle$ and increases the associated behavioral hallmark in task T_2, we denote successful transfer as:

$$p \in \mathcal{P}_{T_1} \;\rightsquigarrow\; p \in \mathcal{P}_{T_2}.$$

4.2 Algorithmic Primitive Extraction & Evaluation

Step 1. Primitive Identification: Geometric Clustering of Latent Representations. We hypothesize that distinct algorithmic primitives are reflected in higher dissimilarities in the model's internal representations, and therefore apply k-means clustering (Lloyd, 1982) to these representations while processing complex self-generated reasoning traces. We chose the k-means algorithm as it provided suitable results on discovering concepts in latent LLM representations in recent investigations (Hawasly et al., 2024; Petukhova et al., 2025; Kopf et al., 2025). For all clustering analyses, we extracted representations from layer 17 of Phi-4-base and used $k = 50$ clusters (we justify these choices in App. G). We fit separate models to TSP, 3-SAT, AIME, and GraphNav and also fit a joint model to TSP+AIME.

Step 2. Primitive Identification: Mapping Clusters to Associated Reasoning Traces. We analyze the primitives associated with the different clusters revealed in step 1 by analyzing the tokens associated with a particular cluster and the context surrounding them. This lets us categorize algorithmic strategies, verification behaviors, and metacognitive patterns and provides direct insight into the models' reasoning processes. To make this process more automatable, we additionally develop an **LLM-driven pipeline for automated labeling** of the reasoning traces associated with different clusters (App. C). The labels map algorithmic primitives in the reasoning trace to latent clusters. We further define a "cluster expressivity" metric (defined as number of unique tokens expressed by a cluster, divided by the radius of that cluster in latent space, App. A.7), reasoning that clusters with higher expressivity will be more likely to be accurate primitives with clear algorithmic meaning. The LLM-driven pipeline and the cluster expressivity together support directly applying our method to novel models and tasks.

Step 3. Primitive Composition: Hierarchical Meta-Clustering and Temporal Clustering. To identify prevalent temporal compositions of these clusters, we then apply spectral clustering (Von Luxburg, 2007) to the matrix of transition probabilities between clusters. This gives rise to a smaller set of meta-cluster which highlight hierarchically structured reasoning steps. To infer the number of meta-clusters, we identify the largest spectral gap, using a minimum of four meta-clusters.

Step 4. Primitive Validation: Primitive Vector Extraction. Finally, we extract primitive vectors associated with particular clusters by adapting the methodology put forward in Todd et al. (2024). Specifically, we extract attention head activations for multiple responses and, for each cluster c, compute the average activations across all tokens belonging to that cluster. We then average the activations over the top $k = 35$ attention heads that reliably carry out in-context learning functions (following the definition in Todd et al. (2024), see App. B.6 for further details).

This yields a candidate primitive vector for each extracted cluster. We validate candidate primitives by defining behavioral hallmarks associated with their proposed computational role and testing whether injecting the extracted primitive vectors results in an increase in those behavioral hallmarks. Beyond validation, we also examine their compositional generalization by testing their arithmetic composition and cross-task transfer, injecting primitive vectors extracted from AIME into the model while it performs TSP.

Algorithmic Fingerprinting. For a set of k clusters and a response j, we extract the relative frequency of tokens assigned to each cluster, $f^{(j)} \in \mathbb{R}^k$. $f^{(j)}$ provides an "algorithmic fingerprint", highlighting differences between the primitives involved in different responses. We measure the algorithmic dissimilarity between responses by the symmetric χ-squared distance between their frequencies,

$$\chi(f, g) := \sum_{i=1}^{k} \frac{(f_i - g_i)^2}{f_i + g_i}. \tag{1}$$

For sets of responses $(f^{(j)})_{j=1}^{n}$, $(g^{(j)})_{j=1}^{m}$, we measure differences in their algorithmic primitives by computing the average frequencies $f = \frac{1}{n}\sum_j f^{(j)}$, $g = \frac{1}{m}\sum_j g^{(j)}$ and then computing the signed χ-squared distance

$$\chi_s(f_c, g_c) := \mathrm{sgn}(f_c - g_c)\frac{(f_c - g_c)^2}{f_c + g_c}, \tag{2}$$

for each cluster c. A large positive difference indicates that the corresponding primitive is more prominent in the responses $(f^{(j)})_{j=1}^{m}$, a large negative difference indicates that it is more prominent in the responses $(g^{(j)})_{j=1}^{m}$. In particular, we compare Phi-4 responses to Phi-4-Reasoning responses and TSP responses to AIME responses.

5 Results

Primitive Tracing and Steering. We first traced primitives by clustering latent representations over entire reasoning traces, interpreting these clusters by identifying the corresponding tokens in the model output. Perhaps surprisingly, we found that despite the simplicity of our approach, many of these clusters were highly interpretable; for example, we discovered a cluster specifically associated with the implementation of a nearest-neighbor heuristic, and another cluster associated with validations, checks, and corrections. Notably, we found that the labels provided by the LLM-driven annotation pipeline were highly consistent with our manual annotations. This validates the accuracy of the LLM-driven pipeline (Table 3).

We then determined the average transition probability between the different clusters. Spectral clustering of this transition matrix revealed that, on TSP, clusters are sequentially composed in a hierarchical and highly stereotyped manner, as most transitions between clusters occurred within the same meta-cluster (Fig. 2a). Notably, different meta-clusters were also responsible for highly interpretable parts of the reasoning, e.g. path generation or computing distances (Fig. 2c).

Next, we validated the role of these primitives by extracting the corresponding primitive vectors and injecting them into Phi-4 during a response generation to TSP. First, we injected two vectors (`nearest_neighbor` and `generate_path`)

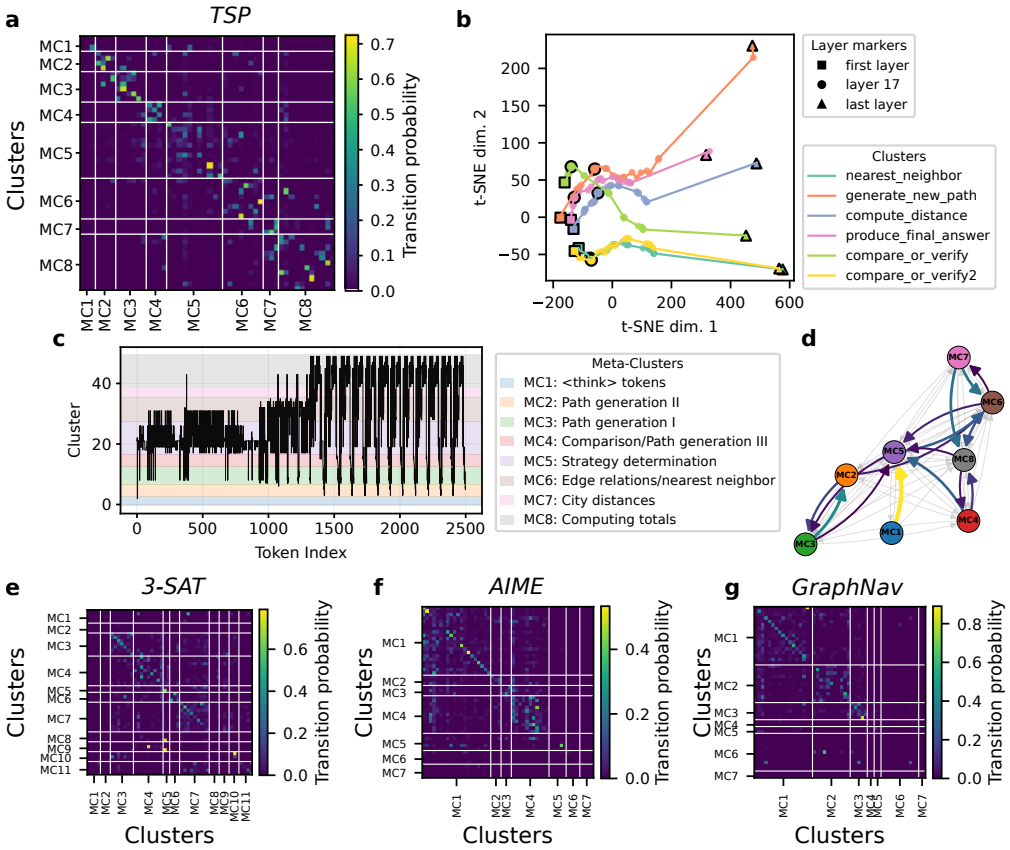

Figure 2: Clusters are organized in a hierarchical manner and meta-clusters are repeatedly traversed throughout the response. **a** Cluster-cluster transition matrix on TSP, organized by meta-cluster (MC) (indicated by the white lines). Most transitions occur within each meta-cluster. **b** Average t-SNE trajectories across different highlighted primitives. **c** Transition between different clusters for an example output. Different meta-clusters are highlighted by colors. The latter half of the response undergoes a cyclical transition. **d** Most common transitions between different meta-clusters reveals frequently occurring cycles. **e-g** Cluster-to-cluster transition matrices structured in terms of the inferred meta-clusters on **e** 3-SAT, **f** AIME, and **g** GraphNav.

Table 2: Effects of primitive vector injection on behavioral hallmarks (% above baseline). For each cell, we identify the maximal effect across all positive magnitudes and intervention layers (10, 13, 15, 17, 20, 30). Bold = strongest effect, underline = second strongest per column. *(%NN paths: Proportion of nearest neighbor paths generated. Dist. comp.: Distance computation. Final ans.: Final answer. #Verif.: number of Verifications in the output. #Comps: number of comparisons in the output. See App. F for detailed definitions.)*

	%NN paths ↑	#Paths ↑	Dist. comp. ↓	Final ans. ↓	#Verif. ↑	#Comp. ↑
`nearest_neighbor`	**+56.1%**	+72.0%	-73.6%	-4.5%	+104.3%	+125.6%
`generate_path`	+4.8%	**+143.9%**	-76.0%	+18.4%	+25.0%	+12.2%
`compute_distance`	+22.2%	+36.5%	**-86.5%**	-47.4%	+198.9%	+79.3%
`final_answer`	+23.2%	+34.8%	-29.8%	**-81.5%**	+82.6%	-37.8%
`compare_verify`	+37.9%	+43.6%	-62.8%	-7.0%	+655.4%	**+1103.7%**
`compare_verify2`	+29.5%	+48.4%	-64.9%	-13.7%	**+706.5%**	+526.8%

into the model throughout the response generation, exploring different layers and magnitudes. We found that `nearest_neighbor` selectively increased the proportion of nearest-neighbor paths generated by the model, whereas `generate_path` increased the number of total generated paths (Fig. 3a,b). This demonstrates that the extracted primitive vectors selectively induce a particular behavior and validates the candidate primitives generated by our clustering.

To expand this investigation, we injected a broader range of candidate primitive vectors into the model after providing example paths and distance computations. For example, injecting the `compute_distance` primitive causes the model